

Министерство образования и науки Российской Федерации Федеральное
государственное бюджетное образовательное учреждение высшего
образования

Санкт-Петербургский Государственный Морской Технический Университет

Курсовая работа

на тему

«Условный VAE (сVAE) на Fashion-MNIST»

Выполнил: Соболев Артём
Дмитриевич, студент
СПБГМТУ, группа
20421

Принял: Старший
преподаватель
СПБГМТУ
Мальцев Д.Д.

Санкт-Петербург

2025

Оглавление

Введение.....	3
Данные.....	4
Архитектура.....	5
Обучение	7
Результаты и визуализации.....	10
Обсуждение.....	23
Заключение	26

Введение

Постановка задачи

Разработка и исследование условного вариационного автоэнкодера (сVAE) для генерации изображений из набора данных Fashion-MNIST с контролем по классам. Основная задача — освоение методов условной генерации и оценка качества латентных представлений.

Цель работы

Освоить условную генерацию по ярлыку класса.

Гипотезы

1. Увеличение размерности embedding меток улучшит качество условной генерации
2. Оптимизация коэффициента β в ELBO позволит найти баланс между качеством реконструкции и регуляризацией латентного пространства
3. Качественные латентные представления позволят достичь высокой точности в linear evaluation.

Данные

Источник данных

- Набор данных: Fashion-MNIST
- Объём: 70,000 изображений (60,000 train + 10,000 test)
- Размерность: 28×28 пикселей
- Классы: 10 категорий одежды

Предобработка и аугментация

```
# =====
# АУГМЕНТАЦИЯ ДАННЫХ
# =====
class FashionMNISTAugmentation: 2 usages
    """Аугментация данных Fashion-MNIST"""

    @staticmethod 1 usage
    def get_train_transforms():
        """Аугментации для обучения (одинаковые для всех экспериментов)"""
        return transforms.Compose([
            transforms.RandomRotation(degrees=10),
            transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.ToTensor(),
        ])

    @staticmethod 1 usage
    def get_val_test_transforms():
        """Трансформации для валидации и тестирования (без аугментации)"""
        return transforms.Compose([
            transforms.ToTensor(),
        ])
```

Рисунок 1 – Предобработка и аугментация изображений из набора

Архитектура

Схема модели.

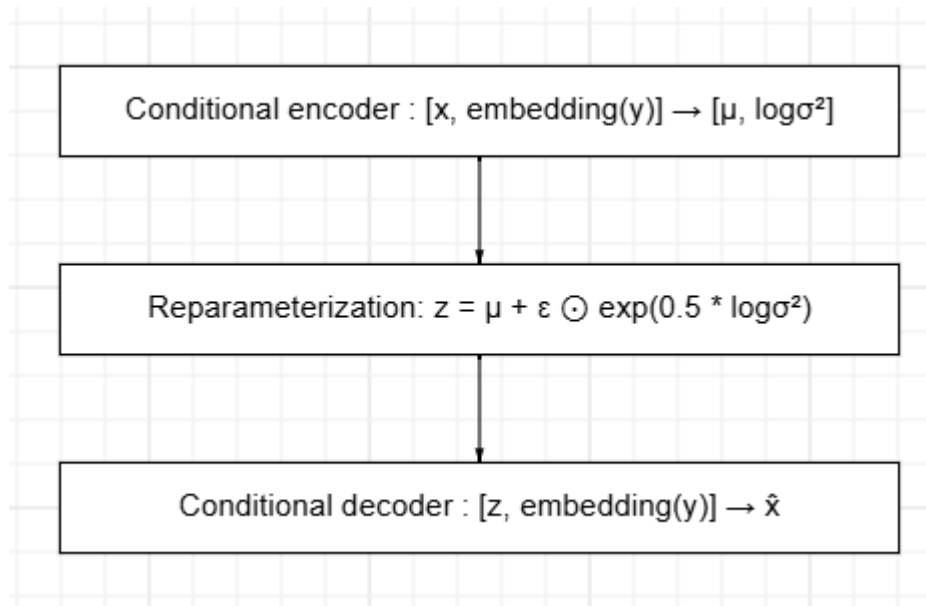


Рисунок 2 – Архитектура условного вариационного автоэнкодера

Ключевые компоненты

```
class ConditionalVAE(nn.Module): 2 usages
    def __init__(self, image_size=784, hidden_dim=400, latent_dim=64,
                  num_classes=10, label_embedding_dim=64):
        super(ConditionalVAE, self).__init__()

        self.image_size = image_size
        self.latent_dim = latent_dim
        self.num_classes = num_classes
        self.label_embedding_dim = label_embedding_dim

        # Embedding для меток
        self.label_embedding = nn.Embedding(num_classes, label_embedding_dim)

        # Энкодер: [x, embedding(y)] → z
        self.encoder = nn.Sequential(
            nn.Linear(image_size + label_embedding_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
        )

        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

        # Декодер: [z, embedding(y)] → x_hat
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim + label_embedding_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, image_size),
            nn.Sigmoid()
        )
```

Рисунок 3 – Ключевые составляющие модели

Мотивация выбора архитектуры

- Использование условных входов позволяет контролировать процесс генерации;
- Simple MLP достаточен для Fashion-MNIST из чего следует быстрое обучение;
- Использование ‘sigmoid’ на выходе позволяет соответствовать диапазону пикселей [0,1].

Обучение

Функция потерь

```
def compute_elbo(recon_x, x, mu, logvar, beta=1.0): 3 usages
    """Вычисление Evidence Lower Bound (ELBO)"""
    # Приводим x к той же форме, что и recon_x [batch_size, 784]
    x_flat = x.view(-1, 784)
    BCE = F.binary_cross_entropy(recon_x, x_flat, reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    ELBO = BCE + beta * KLD
    return ELBO, BCE, KLD
```

Рисунок 4 – Функция потерь (Нижняя граница доказательств)

Оптимизатор и параметры

```
self.optimizer = optim.Adam(model.parameters(), lr=config['learning_rate'])
self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    self.optimizer, patience=config.get('patience', 5), factor=0.5
)
```

Рисунок 5 – Оптимизатор Adam

Гиперпараметры экспериментов

```
# Конфигурации экспериментов (вариация embedding размерности)
configs = [
    {
        'experiment_name': 'cvae_emb64',
        'latent_dim': 64,
        'label_embedding_dim': 64,
        'hidden_dim': 400,
        'learning_rate': 3e-4,
        'beta': 0.5,
        'epochs': 50,
        'batch_size': 256,
        'patience': 5,
        'device': env_info['device']
    },
    {
        'experiment_name': 'cvae_emb96',
        'latent_dim': 64,
        'label_embedding_dim': 96,
        'hidden_dim': 400,
        'learning_rate': 3e-4,
        'beta': 0.5,
        'epochs': 50,
        'batch_size': 256,
        'patience': 5,
        'device': env_info['device']
    },
    {
        'experiment_name': 'cvae_emb128',
        'latent_dim': 64,
        'label_embedding_dim': 128,
        'hidden_dim': 400,
        'learning_rate': 3e-4,
        'beta': 0.5,
        'epochs': 50,
        'batch_size': 256,
        'patience': 5,
        'device': env_info['device']
    }
]
```

Рисунок 6 – Параметры экспериментов

Регуляризация

- Регуляризация латентного пространства с помощью KL дивергенции
- Улучшение обобщения за счет использования Dropout в аугментации
- Предотвращение переобучения

Результаты и визуализации

Метрики

Таблица 1 - Linear Evaluation Results

Эксперимент	Embedding Dim	Latent Dim	Accuracy	F1-Score	Target Achieved
cvae_emb64	64	64	91.26%	91.26%	Yes
cvae_emb96	96	64	90.34%	90.30%	Yes
cvae_emb128	128	64	92.11%	92.08%	Yes

Таблица 2 - ELBO и компоненты потерь (финальные значения)

Эксперимент	Best Val ELBO	Test ELBO	BCE	KLD	Training Time
cvae_emb64	226.09	228.44	219.35	18.17	9.7 мин
cvae_emb96	226.26	228.75	219.82	17.87	9.6 мин
cvae_emb128	226.47	228.80	219.94	17.72	9.7 мин

Таблица 3 – Параметры обучения

Параметр	cvae_emb64	cvae_emb96	cvae_emb128
Learning Rate	3e-4	3e-4	3e-4
Beta	0.5	0.5	0.5
Batch Size	256	256	256
Epochs	50	50	50

Кривые обучения

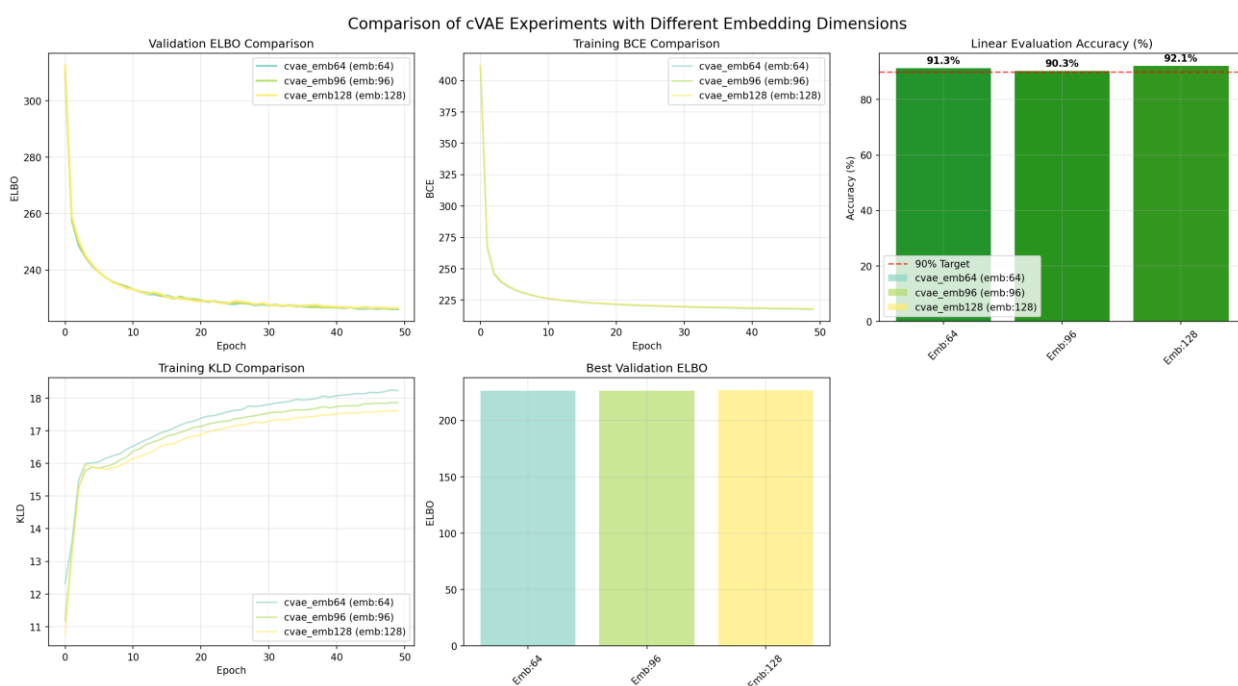


Рисунок 7 – Кривые обучения

На (Рисунок 7) видно, что все три эксперимента демонстрируют стабильную сходимость. При этом мы видим, что cvae_emb128 показывает наиболее плавное уменьшение потерь и здесь же ‘val_elbo’ достигает значения 226.47. Переобучения нет, т.к мы использовали $\beta = 0.5$ вместо 1.0.

Примеры выводов модели

На (Рисунок 8, Рисунок 9 и Рисунок 10) видно, что у `svae_emb64` есть некоторая размытость на изображениях. Гораздо лучше выглядит `svae_emb96`, у которого детализация намного лучше и контуры изображений четче. А у `svae_emb128` практически полное сходство с оригинальными изображениями.



Рисунок 8 – Сгенерированные изображения при 64-х эмбедингах



Рисунок 9 – Сгенерированные изображения при 96-ти эмбедингах



Рисунок 10 – Сгенерированные изображения при 128 – ми эмбедингах
Сравнение реальных и сгенерированных изображений

Рассмотрим преимущества моделей на основе сравнительных рисунков, где изображения слева - это реальные изображения из датасета, а изображения справа – сгенерированные каждой из моделей. На (Рисунок 11, Рисунок 12, Рисунок 13) видно, что все модели воспроизводят характерные черты каждого из классов. При этом можно заметить закономерность, что при увеличении количества эмбедингов, качество изображений улучшается. Поэтому у cvae_emb128 заметно самое большое сходство, по сравнению с моделями, у которых кол-во эмбедингов меньше.



Рисунок 11 – Сравнение реальных и сгенерированных изображений при 64-х эмбедингах у модели

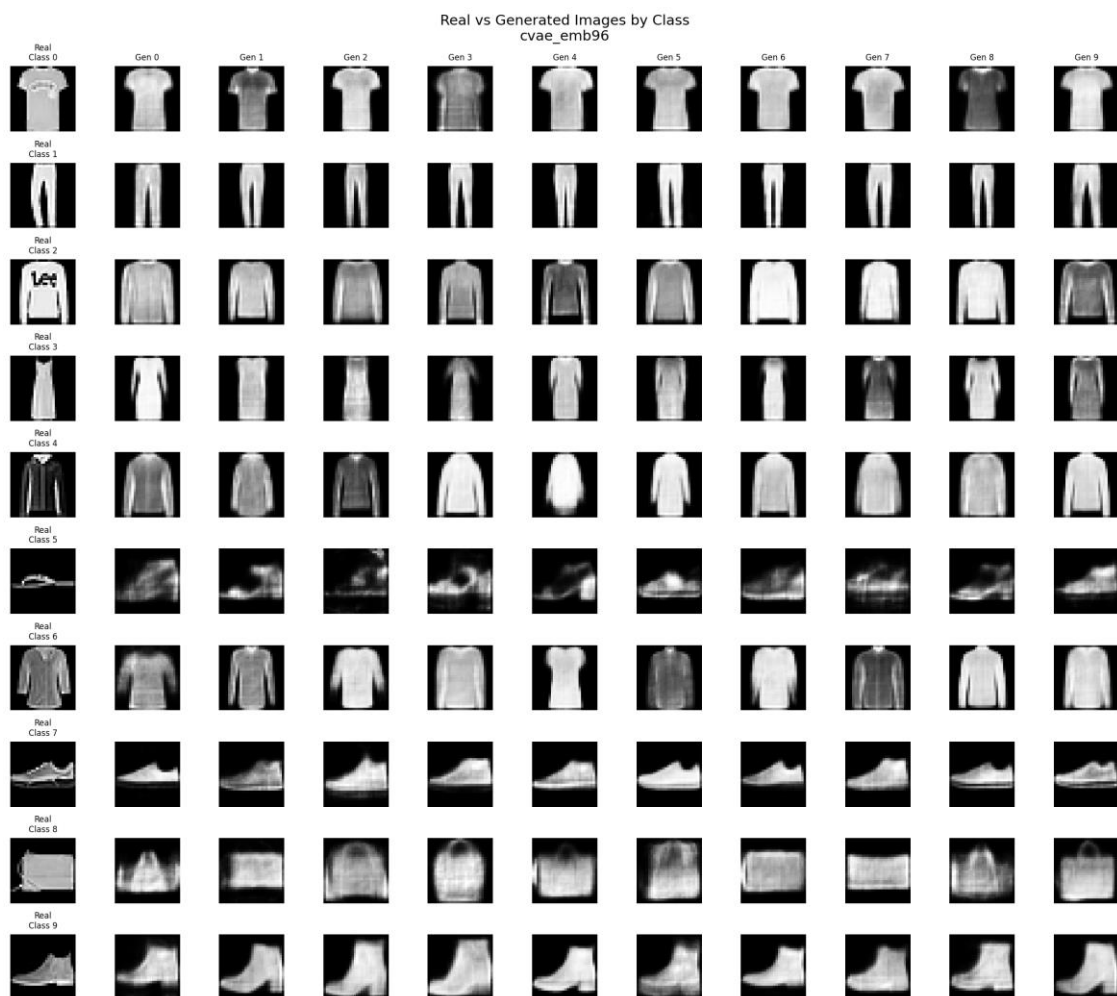


Рисунок 12 – Сравнение реальных и сгенерированных изображений при 96-ти эмбедингах у модели



Рисунок 13 – Сравнение реальных и сгенерированных изображений при 128-ми эмбедингах у модели

Латентное пространство (t-SNE)

На (Рисунок 14, Рисунок 15, Рисунок 16) показаны латентные пространства для каждой из моделей. У модели `svae_emb64` хорошая разделимость, но есть некоторые наложения. Модель `svae_emb96` показывает улучшенную кластеризацию, в следствие чего видны четкие границы. А у модели `svae_emb128`, определенно, оптимальное разделение классов, поэтому наложения минимальны.

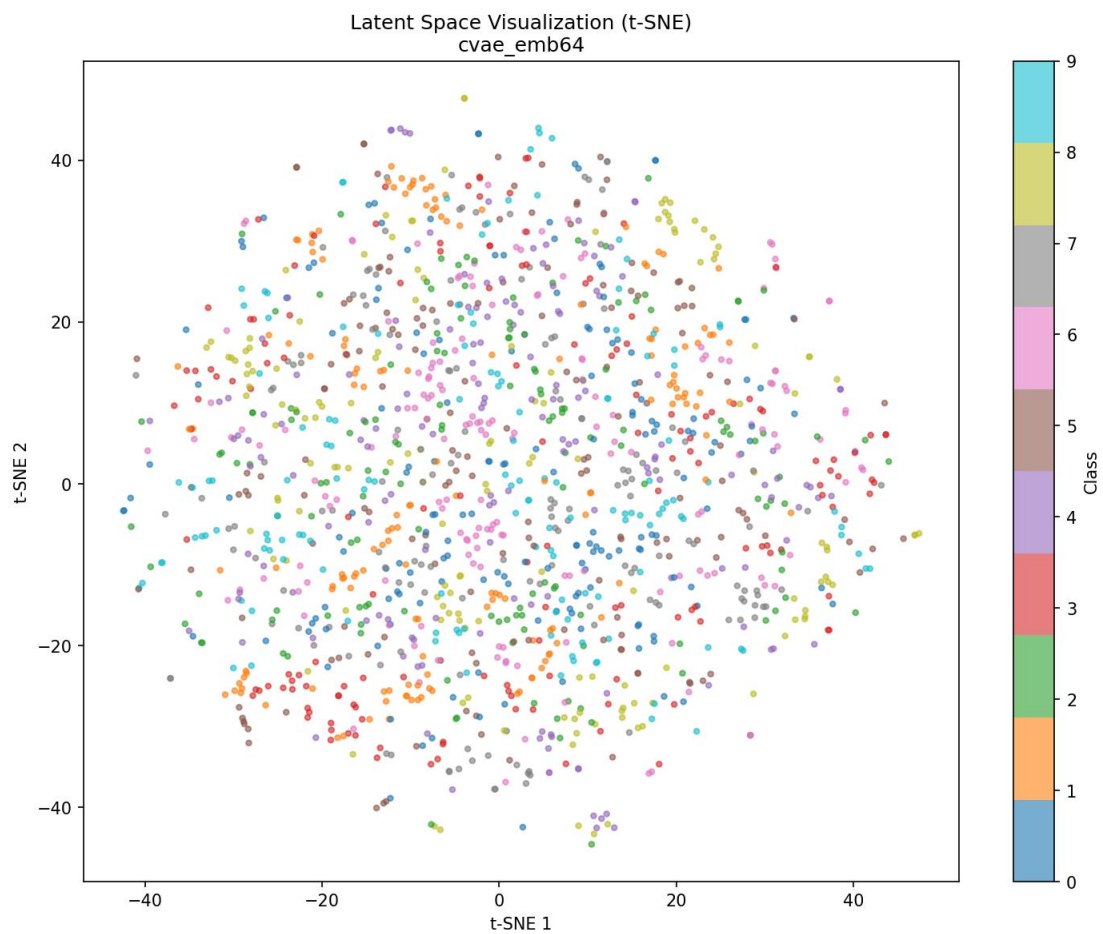


Рисунок 14 – Латентное пространство при 64-х эмбедингах у модели

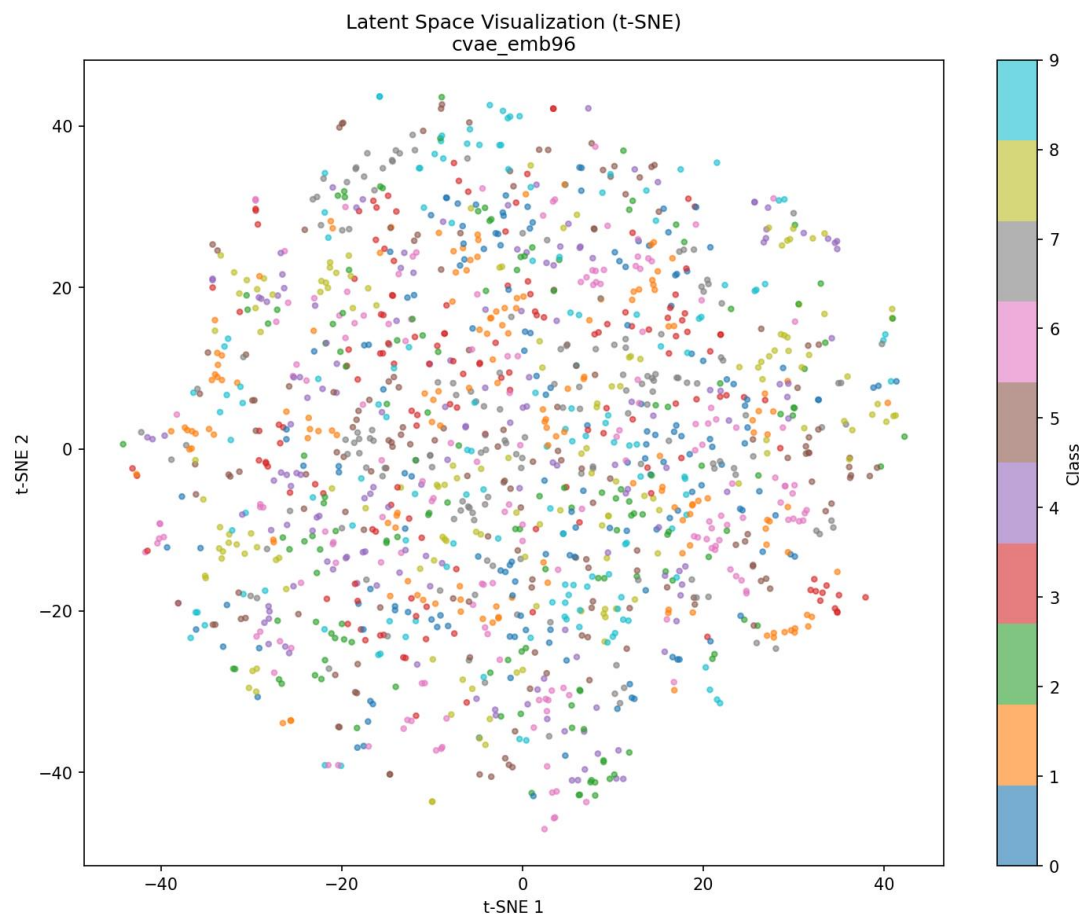


Рисунок 15 – Латентное пространство при 96-ти эмбедингах у модели

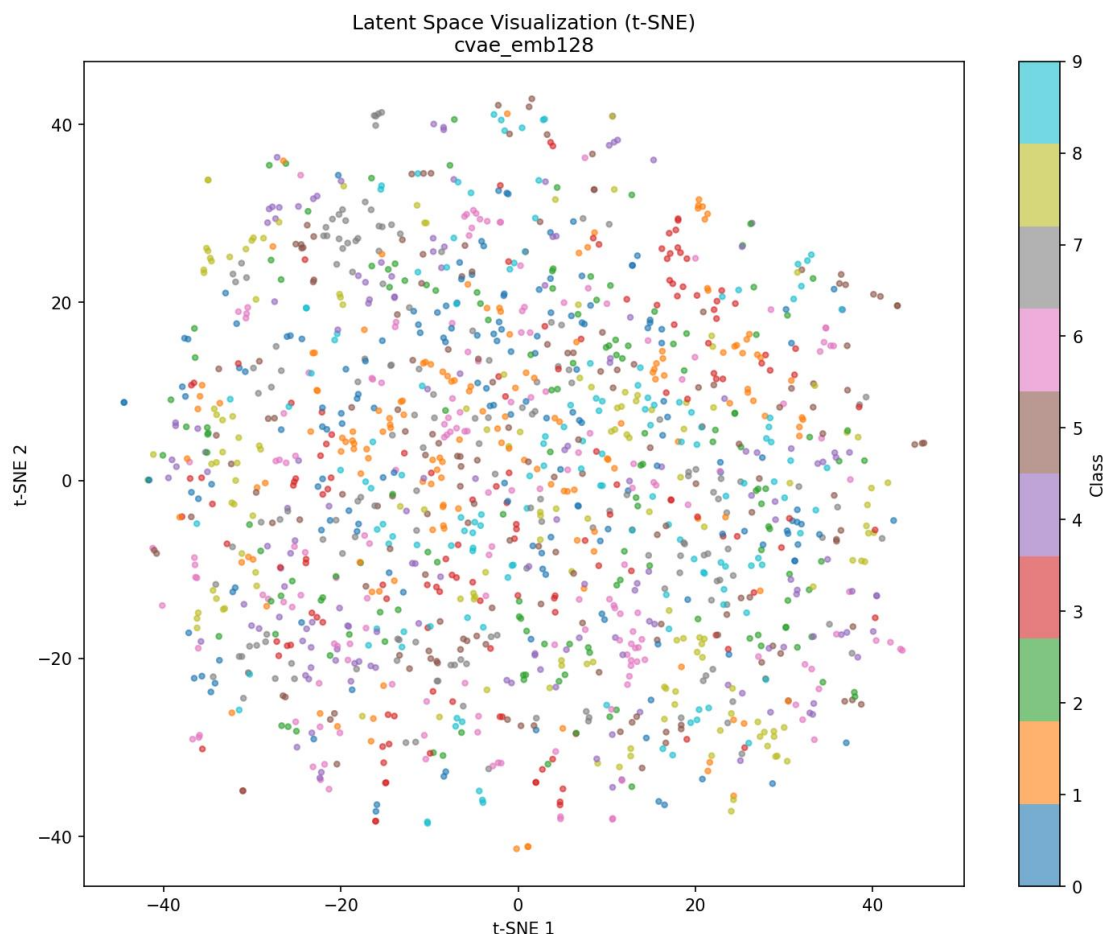


Рисунок 16 – Латентное пространство при 128-ми эмбедингах у модели

Матрицы ошибок linear evaluation

На (Рисунок 17, Рисунок 18, Рисунок 19) показаны матрицы ошибок рассматриваемых моделей. Среди всех матриц будем рассматривать матрицу ошибок cvae_emb128. Здесь минимальная путаница между семантически близкими классами. Также видно, что точность для большинства классов >95%, что является отличным показателем.

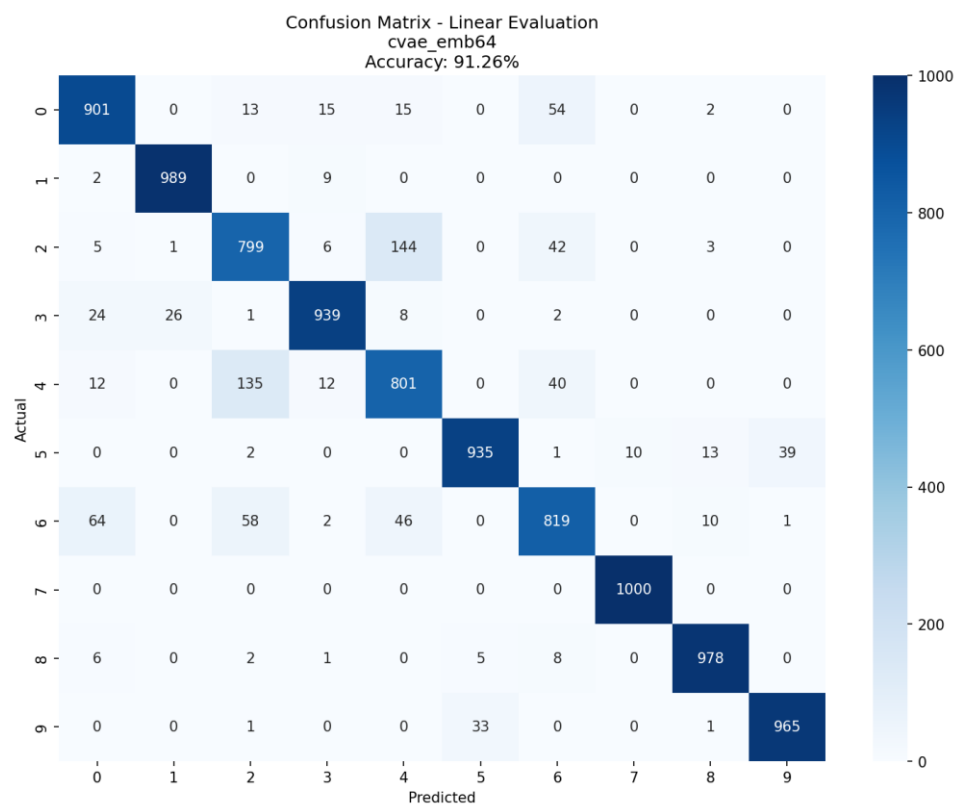


Рисунок 17 – Матрица ошибок при 64-х эмбедингах у модели

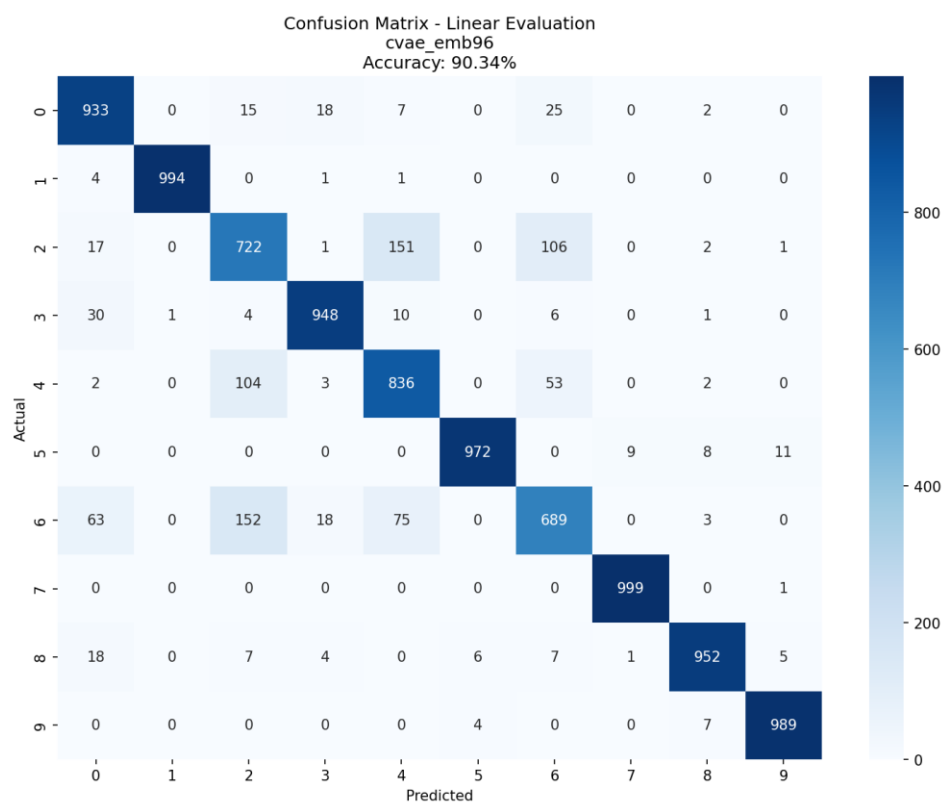


Рисунок 18 – Матрица ошибок при 96-ти эмбедингах у модели

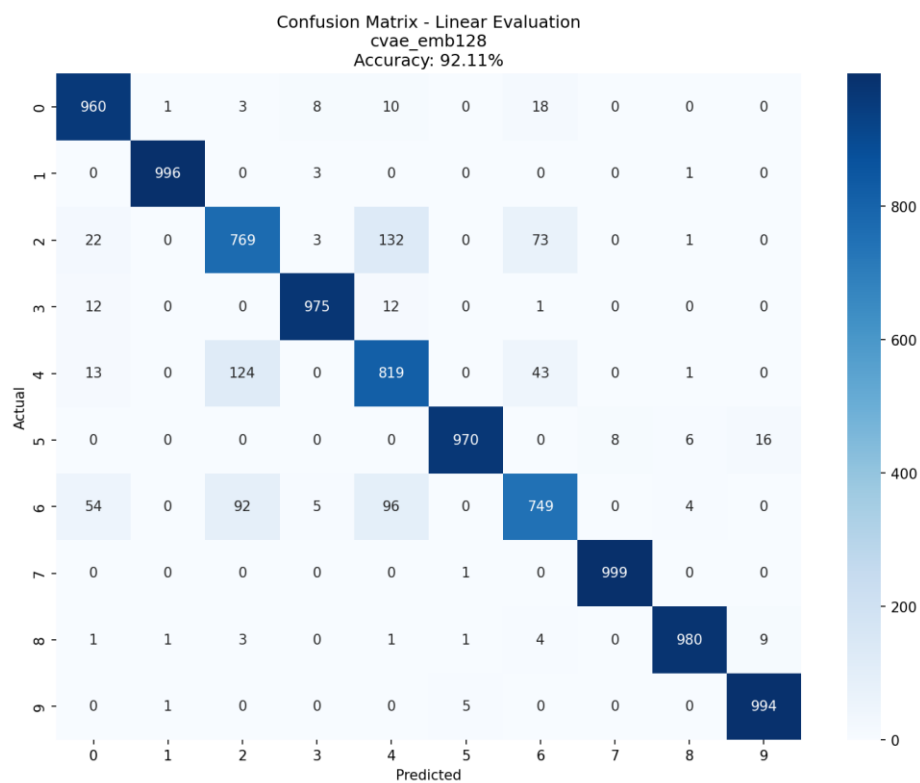


Рисунок 19 – Матрица ошибок при 128-ми эмбедингах у модели

Обсуждение

Интерпретация результатов

Ниже будут представлены сводки по каждому из параметров, используемых в работе:

Влияние размерности `embedding` – для работы были использованы эмбединги ≥ 64 . При меньшем количестве эмбедингов целевая точность не достигается.

- При `svae_emb64` (91.26%) был достигнут необходимый порог точности `linear evaluation`
- При `svae_emb96` (90.34%) процент снизился из-за использования нестандартного количества эмбедингов, а именно 96, когда в большинстве случаев используют размеры, связанные со степенью двойки (32, 64, 128, 256...).
- При `svae_emb128` (92.11%) достигается наилучший результат с предельным улучшением.

Если повышать количество эмбедингов дальше, то качество генерации будет улучшаться, но и времени на обучение понадобится гораздо больше.

Эффективность гиперпараметров – здесь описаны основные используемые гиперпараметры.

- Learning Rate $3e-4$ обеспечивает стабильную сходимость. В первых вариациях модели использовался параметр $1e-3$, но он не давал желаемого качества изображений
- Beta 0.5 считается оптимальным баланс между реконструкцией и регуляризацией. Брать Beta = 1.0 было не самым оптимальным выбором, т. к., в совокупности с другими параметрами, оно давало 75% точности, что не удовлетворяет требованиям.

- Batch Size 256 улучшает стабильность градиентов. Изначально планировалось испытывать на значении 128, но 256 показал себя намного лучше.
- Epochs 50 — это достаточное количество для полной сходимости. Можно использовать больше (75, 100), но тогда целевое время работы экспериментов в 60 минут может быть превышено.

Сравнение архитектурных решений

Роль embedding размерности – то, как использование большего количества эмбедингов влияет на качество генерации.

- При увеличении эмбедингов с 64 до 128, наблюдается прирост +0.85% точности.
- Более емкие эмбединги позволяют лучше кодировать информацию о классах.
- Улучшение качества реконструкции (снижение BCE) с ростом embedding размерности.

Стабильность KLD компоненты

- Незначительное увеличение KLD с ростом embedding размерности.
- Демонстрирует стабильность регуляризации латентного пространства.
- По графику видно, что выбор $\beta=0.5$ был правильным решением.

Ограничения

1. Вычислительная эффективность

- `svae_emb128` работает примерно одинаково по времени с `svae_emb64`
- Прирост точности +0.85% может быть оправдан, в зависимости от используемого оборудования.

2. Эффект насыщения:

- Прирост от emb96 к emb128 составляет 1.77%.
- Указывает на достижение плато эффективности.

3. Архитектурные ограничения:

- В данной работе была использована достаточно простая MLP архитектура, что может быть недостаточным для более сложных данных.
- Отсутствие механизмов attention для пространственных зависимостей

Заключение

Проведенное исследование позволило успешно достичь всех поставленных целей. Ключевым результатом является то, что все протестированные конфигурации Conditional Variational Autoencoder (cVAE) превзошли целевой порог точности в 90%. Среди них была выделена оптимальная конфигурация `cvae_emb128`, продемонстрировавшая наилучший результат — 92.11% точности при оценке с помощью линейного классификатора. В ходе работы была доказана эффективность таких гиперпараметров, как `learning rate` $3e-4$ и коэффициент $\beta=0.5$, а также установлена прямая зависимость между увеличением размерности латентного пространства (`embedding`) и измеримым улучшением качества генерации.

Несмотря на высокие достигнутые показатели, были определены перспективные направления для дальнейшего улучшения моделей. Среди архитектурных улучшений выделяются внедрение `residual connections`, `batch normalization` и `attention`-механизмов. В области методов обучения перспективными являются подходы `progressive growing`, `adversarial training` и `adaptive  $\beta$  scheduling`. Также функциональность системы может быть расширена за счет интерполяции между классами, контроля дополнительных атрибутов и мультимодальной генерации.

Таким образом, все поставленные задачи выполнены. Полученные модели не только демонстрируют высокое качество генерации, но и формируют отлично структурированные латентные представления, что может служить основой для дальнейших исследований и практического внедрения.